

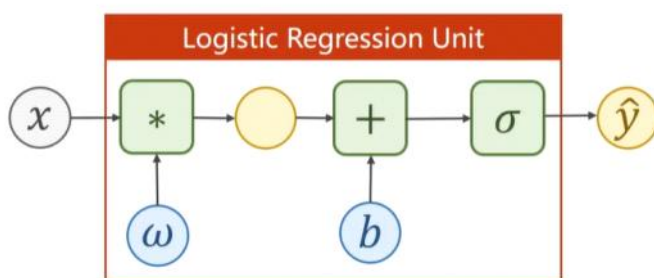
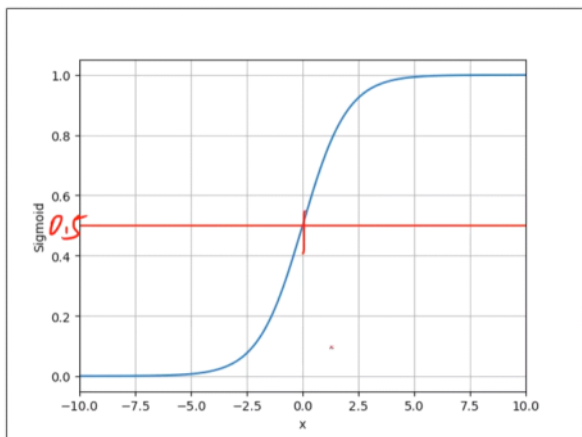
【pytorch06】逻辑斯蒂回归

2026年9月8日 16:53

回归 vs 分类：分类任务中，模型输出的是「输入属于某类的概率」。

Sigmoid / Logistic 函数：

$\sigma(x) = 1 / (1 + e^{(-x)})$ ，把任意实数映射到 (0, 1)。



Logistic 回归模型： $\hat{y} = \sigma(wx + b)$ （线性层后接 Sigmoid）。

二分类损失 = 二元交叉熵 BCE： $\text{loss} = -(y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y}))$ **区别于线性模型的MSE**

Mini-Batch 版本： $\text{loss} = -(1/N) \cdot \sum [y_i \cdot \log(\hat{y}_i) + (1-y_i) \cdot \log(1-\hat{y}_i)]$

实现要点：

模型：`nn.Linear(1,1) + F.sigmoid(...)`;

损失：`torch.nn.BCELoss(size_average=False)`;

优化器：**SGD**。

数据集引入：MNIST（手写数字，10类）、CIFAR-10（10类），通过 `torchvision.datasets` 加载。

代码：

```
# 第 06 课 · 逻辑回归 (Logistic Regression)
# 内容：二分类。模型输出「属于某类的概率」
# 模型：  $y = \sigma(wx + b)$ ,  $\sigma$  为 Sigmoid
# 损失：BCE（二元交叉熵）
# 依赖：numpy, matplotlib（画图）
```

```
import torch
```

```

import torch.nn.functional as F
import numpy as np
import matplotlib.pyplot as plt

# ----- 1. 准备数据集 -----
x_data = torch.Tensor([[1.0], [2.0], [3.0]])
y_data = torch.Tensor([[0], [0], [1]]) # 二分类标签 0/1

# ----- 2. 用类设计模型 -----
class LogisticRegressionModel(torch.nn.Module):
    def __init__(self):
        super(LogisticRegressionModel, self).__init__()
        self.linear = torch.nn.Linear(1, 1)

    def forward(self, x):
        y_pred = F.sigmoid(self.linear(x)) # 与05课的不同：线性层后接 Sigmoid，将输出值控制在0和1之间
        return y_pred

model = LogisticRegressionModel()

# ----- 3. 构造损失函数和优化器 -----
criterion = torch.nn.BCELoss(size_average=False) # 二元交叉熵损失，于之前的MSE不同
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

# ----- 4. 训练循环 -----
for epoch in range(1000):
    y_pred = model(x_data)
    loss = criterion(y_pred, y_data)
    print(epoch, loss.item())

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

# ----- 画图观察分类概率曲线 -----
x = np.linspace(0, 10, 200)
x_t = torch.Tensor(x).view((200, 1))
y_t = model(x_t)
y = y_t.data.numpy()
plt.plot(x, y)
plt.plot([0, 10], [0.5, 0.5], c='r') # 0.5 分界线
plt.xlabel('Hours')
plt.ylabel('Probability of Pass')
plt.grid()
plt.show()

```